

Documentação Técnica: Arquitetura ZoyBlocks Cozmo (Modo VM)

1. Visão Geral do Sistema

A IDE **ZoyBlocks Cozmo** é uma plataforma de programação visual baseada em Blockly que permite o controle em tempo real do robô Cozmo. O diferencial técnico desta arquitetura é o uso de uma **Máquina Virtual (VM)** em Node.js para interpretar o código JavaScript gerado, servindo como uma ponte entre a interface do usuário e o hardware físico.

2. A Camada de Interface (Renderer)

A interface é o ponto de entrada do usuário, onde a lógica de programação é construída.

- **Ferramenta:** Google Blockly.
- **Toolbox Estático (XML):** As categorias de blocos (Movimentos, Lógica, Loops) são definidas diretamente no arquivo `home.html`. Isso garante que apenas os blocos necessários para o Cozmo e a lógica básica estejam disponíveis.
- **Geração de Código:** Ao clicar em "Executar", o Blockly converte os blocos visuais em uma string de texto JavaScript puro.

3. O "Cérebro" da IDE: A Máquina Virtual (VM)

Diferente de sistemas que compilam o código, esta IDE utiliza o módulo `node:vm` para criar um ambiente de execução isolado e controlado.

Por que usar uma VM?

- **Sincronização (Awaited Execution):** O JavaScript nativo é rápido demais para o hardware. A VM permite o uso de `async/await`, garantindo que o comando "Mover" termine no robô antes que o próximo comando comece.
- **Segurança (Sandbox):** O código do usuário roda em um ambiente protegido. Se o usuário criar um loop infinito, o parâmetro `timeout: 30000` (30 segundos) na VM encerra a execução automaticamente para não travar o computador.
- **Injeção de Contexto:** A VM recebe um "mapa" de funções (como `mover`, `virar`, `parar`) que traduzem o código JavaScript em ações reais.

4. O Fluxo de Dados (O caminho da informação)

O processo segue uma sequência rigorosa de 5 etapas:

1. **Bloco Visual:** O usuário encaixa os blocos no `home.html`.
2. **JavaScript (String):** O `home.js` captura o código gerado pelo Blockly.
3. **VM (Execução):** O `blockly-service.js` executa o código. Ao encontrar um comando como `mover(50, 1)`, ele aciona a lógica de comunicação.
4. **JSON (Tradução):** O comando é convertido em um objeto JSON, por exemplo: `{"cmd": "move", "speed": 50, "duration": 1}`.
5. **Python (Ação Física):** O `main.js` envia esse JSON via `stdin` para o processo `cozmo_server.py`, que finalmente move os motores do robô.

5. Componentes Principais do Código

- **main.js:** Gerencia o ciclo de vida do aplicativo Electron e inicia o processo Python do Cozmo.
- **blockly-service.js:** Contém a lógica da VM e o mapeamento de tempo (ex: esperar 1.5s para um giro de 90 graus).
- **home.js:** Controla o carregamento de scripts, a injeção do workspace e a interface de logs (Terminal).

6. Conceitos Chave para Estudo (Flashcards)

- **IPC (Inter-Process Communication):** Como o Front-end fala com o Back-end no Electron.
- **Spawn de Processos:** A técnica de iniciar um script Python de dentro de um programa Node.js.
- **Interpretador vs Compilador:** Por que rodar código via VM no PC é mais rápido para testar do que gravar um firmware no chip.

Dica para o NotebookLM:

Ao carregar este documento, peça para o NotebookLM criar um "Plano de Aula para Desenvolvedores Júnior" focando no capítulo 4 (Fluxo de Dados), pois é onde a integração entre JavaScript, JSON e Python acontece.